1. Análisis de Tecnologías Front-end:
Clasifica las siguientes opciones en Alternativas Históricas Obsoletas, Estándar Nativo Universal o Dialectos/Superconjuntos con Transpilación:
○ Adobe Flash (ActionScript) → obsoleta
○ TypeScript → Dialecto con transpilacion
○ Java Applets →obsoleta
○ JavaScript (ECMAScript) → estandar nativo universal
○ JSX → dialecto con trnaspilacion
○ VBScript → obsoleta

2. Justificación de Arquitectura:
Un equipo de desarrollo está debatiendo la arquitectura para una nueva aplicación web financiera.
○ Explica dos ventajas técnicas de utilizar JavaScript nativo en el cliente para reducir el uso de recursos en el servidor.
Dos ventajas técnicas de usar JS nativo en el cliente para reducir recursos del servidor:
Delegación del cálculo en el cliente: validaciones, formateo de datos, ordenaciones y cálculos de interfaz se ejecutan con la CPU y RAM del dispositivo del usuario, no del servidor. El servidor queda libre para tareas críticas.
Menos tráfico de red: se transfieren solo datos (JSON) en lugar de documentos HTML completos ya renderizados. Además, al validar en el cliente se evitan peticiones innecesarias al servidor.

○ Define el concepto de Zero Plugins y menciona qué problema histórico

JavaScript es interpretado de forma nativa por los motores de los navegadores modernos (V8, SpiderMonkey, JavaScriptCore) sin instalar ningún complemento. El problema histórico que resuelve es la dependencia de plugins externos pesados, que el usuario debía descargar, instalar y mantener (Applets de Java, Adobe Flash, VBScript): eran lentos, consumían batería y recursos, y suponían un grave agujero de seguridad.

3. Matriz de Selección de Dialectos:
A partir de los requisitos de tres proyectos distintos, indica la tecnología recomendada (Vanilla JS, TypeScript o JSX) y justifica tu respuesta en base al tipo de proyecto y la fase de detección de errores:
○ Proyecto A: Una plataforma empresarial bancaria con un equipo de 30 desarrolladores que requiere validación estática de datos en tiempo de desarrollo.
Typescript →Tipado fuerte y estático: los errores se detectan en desarrollo/compilación previa, antes de llegar a producción. Imprescindible en equipos grandes y código crítico.

○ Proyecto B: Un script ligero de 50 líneas para manipular el DOM e integrar un widget flotante en un sitio web estático.
JS vainilla→No hay compilación ni herramientas extra; para algo tan pequeño la detección de errores en runtime es asumible.

○ Proyecto C: Una interfaz de usuario interactiva basada en componentes reutilizables utilizando ReactJS.
JSX → Es la sintaxis de plantillas de React; se compila (Babel) a llamadas JS convencionales al construir el árbol de componentes.

4. ECMAScript y Transpilación:
Define la relación entre ECMAScript y JavaScript. Explica qué papel desempeña un transpilador (como Babel) cuando se utiliza sintaxis moderna de ES6/ES2015 en entornos corporativos que deben mantener compatibilidad con navegadores antiguos.
- ECMAScript es la especificación/estándar (norma ECMA-262 de ECMA International): define la sintaxis, los tipos y el comportamiento que deben cumplir los motores.
- JavaScript es la implementación real de ese estándar en los navegadores.
- El transpilador (Babel, o tsc de TypeScript) traduce la sintaxis moderna (ES6/ES2015+, o TypeScript) a JavaScript estándar antiguo (ES5) equivalente, que los navegadores viejos sí entienden. Así se puede escribir código moderno sin perder compatibilidad en entornos corporativos.

5. Validación Sintáctica de Identificadores:
Analiza la siguiente lista de nombres de variables. Indica cuáles son válidos y cuáles son inválidos según las reglas de JavaScript. Para los inválidos, especifica el motivo exacto de la infracción:
○ 1erUsuario → Invalido, empieza por dígito
○ _totalFactura →válido, empieza con guión bajo

○ $elementoDOM → válido, empieza con $
○ precio-final → invalido, guión en medio
○ class → invalido, palabra reservada
○ montoTotal2 → válido, camelCase
○ nombre usuario → invalido, incluye un espacio
○ function → invalido, palabra reservada

6. Detección de Errores por Sensibilidad a Mayúsculas (Case-sensitive):
El siguiente código JavaScript presenta múltiples errores sintácticos o de referencia debido al mal uso de mayúsculas y minúsculas. Identifica los 4 errores presentes y reescribe el fragmento corregido:

```
function calcularDescuento(precioBase) {
  const porcentaje = 0.15;
  if (precioBase > 100) {
    alert("Descuento aplicado");
    return precioBase * porcentaje;
  }
  return 0;
}

console.log(calcularDescuento(150)); // 22.5
```

7. Aplicación de Convenciones Profesionales: Refactoriza los nombres de las siguientes entidades aplicando la convención de nomenclatura adecuada (camelCase, PascalCase, UPPER_SNAKE_CASE, $ o _):
○ Una variable para almacenar el nombre del cliente actual. → nombreCliente
○ Una constante global inmutable para el número máximo de reintentos de conexión. → MAX_REINTENTOS
○ Una clase que representa a un usuario registrado en el sistema. → UsuarioRegistrado
○ Una variable que almacena una referencia directa a un botón del DOM. → $botonEnviar
○ Una propiedad de un objeto con intención de uso privado para guardar el token de sesión. → _tokenSesion

8. Análisis del Comportamiento del ASI (Inserción Automática de Punto y Coma):
Observa el siguiente bloque de código:

```
function crearConfiguracion() {
  return
  {
    modo: "oscuro",
    puerto: 8080
  };
}

const config = crearConfiguracion();
console.log(config);
```

● a) ¿Qué valor se imprimirá exactamente en la consola del navegador al ejecutar este script? → undefined

● b) Explica paso a paso cómo interviene el mecanismo ASI del motor de JavaScript para producir dicho resultado.
el motor aplica ASI (Automatic Semicolon Insertion). Al ver return seguido de un salto de línea, inserta automáticamente un ; justo después de la palabra reservada:

```
return;   // <-- el ASI cierra la sentencia aquí
{ modo: "oscuro", puerto: 8080 };  // bloque suelto, NO se devuelve
```

Como el return queda «vacío», la función devuelve undefined. La llave { ... } de debajo ya no forma parte del return y se ejecuta como un bloque aislado.

● c) Reescribe el código corregido aplicando las buenas prácticas marcadas en el temario.

```
function crearConfiguracion() {
  return {
    modo: "oscuro",
    puerto: 8080
  };
}

const config = crearConfiguracion();
console.log(config);
```

9. Verificación en Consola y Ámbito:
Indica qué salida se obtiene en la consola al ejecutar de forma secuencial las siguientes líneas y explica el comportamiento del motor respecto a identificadores independientes:

```
let totalVentas = 500;
let TotalVentas = 1200;

console.log(totalVentas); → 500
console.log(TotalVentas); → 1200
```